(a) Chain of Thought

```
Q: Roger has 5 balls.  He buys 2 more packs, each with 3.  How many balls does he have now?

Roger starts with 5 balls.
2 packs of 3 balls is 6.
5 + 6 = 11.

A: 11
```

(b) Program of Thoughts

```
Q: Roger has 5 balls.  He buys 2 more packs, each with 3.  How many balls does he have now?

num_balls = 5
num_balls += 2 * 3
answer = num_balls

A: 11
```

(c) ScratchPad

```
Q: Roger has 5 balls.  He buys 2 more packs, each with 3.  How many balls does he have now?

num_balls = 5   state:  {num_balls = 5}
num_balls += 2 * 3   state:  {num_balls = 11}
answer = num_balls   state:  {answer = 11}

A: 11
```

(d) Chain of Code Generation (Ours)

```
Q: I have an orange, a violin, two peaches, an apple, a pepper, and three plums.  How many fruits do I have?

objects = {"orange":  1, "violin":  1, "peaches":  2, "apple":  1, "pepper":  1, "plum":  3}
num_fruits = 0
for object in objects:
  object_is_fruit = is_fruit(object)
  if object_is_fruit:
    num_fruits += objects[object]
answer = num_fruits
```

(e) Chain of Code Execution (Ours)

```
Q: I have an orange, a violin, two peaches, an apple, a pepper, and three plums.  How many fruits do I have?

objects = {"orange":  1, "violin":  1, "peaches":  2, "apple":  1, "pepper":  1, "plum":  3}
delta state:  {objects = {'orange':  1, 'violin':  1, ...}}
num_fruits = 0
delta state:  {num_fruits = 0}
for object in objects:
delta state:  {object = 'orange'} # updated for each loop
  object_is_fruit = is_fruit(object)
delta state:  {object_is_fruit = True}
  if object_is_fruit:
delta state:  {}
    num_fruits += objects[object]
delta state:  {num_fruits = 1}
answer = num_fruits
delta state:  {answer = 7}

A: 7
```

Figure 3. **Previous reasoning methods:** To solve advanced problems, (3a) Chain of Thought prompting breaks the problem down into intermediate steps, (3b) Program of Thoughts prompting writes and executes code, and (3c) ScratchPad prompting simulates running already written code by tracking intermediate steps through a program state. **Our reasoning method:** Chain of Code first (3d) generates code or psuedocode to solve the question and then (3e) executes the code with a code interpreter if possible, and with an LMulator (language model emulating code) otherwise. Blue highlight indicates LM generation, red highlight indicates LM generated code being executed, and purple highlight indicates LMulator simulating the code via a program state in green.

2. It leverages the ability of language models to code, a particular strength of recent language models due to the high quality data available.

3. It inherits many of the benefits of reasoning code, both the formal yet expressive structure of code (e.g., Turing completeness) and powerful computational tools available to code (whether simply multiplying two numbers, calculating $\sqrt[5]{12121}$, or simulating physics).

4. It inherits many of the benefits of techniques that reason via intermediate steps, such as Chain of Thought. These techniques enable the language model to use more computation when necessary to solve a problem as well as provide more interpretability.

Empirically, we observe in Section 3 that these benefits results in significant improvements in reasoning performance over a variety of challenging tasks.

3. Experimental Evaluation

We select challenging problems requiring varied types of reasoning, whether arithmetic, commonsense, or symbolic reasoning tasks, to answer the following questions:

1. How well does CoC perform across a variety of tasks?
2. Which types of problems does CoC perform best?
3. How does each aspect of CoC affect performance?
4. How does CoC scale with model size?
5. How does CoC perform as a general-purpose reasoner, with prompt examples from different problems rather than the same problem (which we term cross-task prompting)?
6. How can CoC be used with instruction tuned chat models?
7. How robust CoC is against prompt variation?
8. Can CoC be applied beyond language reasoning tasks?

We first discuss the approaches, ablations, and baselines considered in Section 3.1, then the tasks considered in Section 3.2, and finally the results in Section 3.3.

3.1. Baselines and Ablations

We consider our main method to be **CoC (Interweave)**, also referred to as **CoC (Ours)**, though we also propose two variants with simpler implementation and modestly lower performance: **CoC (try Python except LM)** and **CoC (try Python except LM state)**. These two variants attempt to run the entire generated code with Python (rather than line by line) and if it fails, simulate the code execution with the LMulator, outputting a final answer or an intermediate state trace, respectively. We also perform the following ablations, some of which are comparable to previous work as noted. In **CoC (Python)** Python is used to run the entire generated code and if the code is not executable, it is marked as failure – this can be thought of as a comparison to Program of Thoughts (Chen et al., 2022) or Program-aided language models (Gao et al., 2023). We note that in many cases this baseline is particularly challenged, as writing executable code for some of the reasoning problems becomes nearly impossible (e.g., writing code to judge if a phrase is sarcastic), but one may focus on the results for Algorithmic only tasks for a more fair comparison. In **CoC (LM)** the code is interpreted by an LMulator outputting the final answer, and in **CoC (LM state)** the code is interpreted by an LMulator outputting a state trace of intermediate steps – this can be thought of as ScratchPad prompting for reasoning (Nye et al., 2021). Note, the last two ablations do not leverage the Python interpreter.

We also compare against the following baselines. In **Direct** question answering the LM simply responds to the question with a final answer. In Chain of Thought prompting **(CoT)** the LM uses intermediate steps to solve the task; we use CoT as our standard prompt technique for the field of substep prompting (Kojima et al., 2022; Zhou et al., 2022a) as prompts are readily available.

3.2. Tasks

We consider a subset of challenging tasks from BIG-Bench (Srivastava et al., 2022) called BIG-Bench Hard (BBH) (Suzgun et al., 2022) to ensure we are solving the most challenging tasks. These tasks were specifically selected for their difficulty for language models and the datasets provides human-rater baselines and a set of Chain of Thought prompts. The 23 tasks require semantic reasoning (e.g., "Movie Recommendation"), numerical reasoning (e.g., "Multi-Step Arithmetic"), and a combination of both (e.g., "Object Counting"). As such they enable us to study the efficacy of CoC across varied problems, not just those that coding is a natural fit for. Several prompts are shown in Figure A1. We also show results for the grade-school math (GSM8K) benchmark (Cobbe et al., 2021) in Section A.2, although we find that these problems are primarily solved algorithmically alone through code.

These tasks are evaluated with **few-shot prompting**, whereby three examples from the same problem family are provided as context. We also introduce a new evaluation setting, **cross-task prompting**, whereby three examples of *different* problems are provided as context. As such, the language model has in-context examples of the *format* of reasoning, but isn't provided explicit instructions on *how* to reason. We see this as an indicative signal for a general-purpose reasoner, which in many real-world applications (e.g., chatbots) would be asked to reason across a wide variety of tasks.

The models used herein include the OpenAI family of models: `text-ada-001`, `text-baggage-001`, `text-curie-001`, and `text-davinci-003` (in plots we denote these as a-1, b-1, c-1, and d-3). We also consider PaLM-2's code finetuned variant (Chowdhery et al., 2022; Google et al., 2023). For instruction tuned models, we compare to recent variants of GPT (`gpt-3.5-turbo` and `gpt-4`) with the chat completion mode run in October 2023 and January 2024. The results below are using the `text-davinci-003` model unless otherwise stated.

3.3. Results

Question 1: Overall Performance. The overall performance of CoC is shown in Figure 2 and Table 1 (with full results in Table A1). We see that CoC outperforms other approaches, both in the number of tasks it exceeds the human baseline and in the overall amount that it exceeds the baseline. Indeed, CoC's 84% is SoTA to the best of our knowledge (Gemini Team, 2023). In fact, when combined with `gpt-4`, CoC achieves 91% (see Table A4). In several tasks CoC vastly outperforms the human baseline and other methods, achieving nearly 100% – generally for these tasks the result is complicated in language but trivial in code (e.g., a task from multi-step arithmetic `Q:` $((-3+5\times8\times-4)-(9-8\times-7))=$). We also observe that CoT outperforms the human baseline on a number of tasks, while the Direct answer fares poorly.

Question 2: Problem Type. Figure 4 breaks the results down by problem type; the task labels are shown in Table A1. First, we isolate problems that are primarily algorithmic or primarily natural language (these categories were identified in (Suzgun et al., 2022)). We see that on algorithmic tasks, CoC performs particularly well, while on natural language tasks CoC performs on par with CoT. This is particularly encouraging, because one may expect these language oriented tasks to be a worse fit for code. The key is that our method offers the flexibility of using a LMulator to simulate the output of code execution, retaining the semantic reasoning capabilities of LMs for natural language problems.

Figure 4 additionally breaks the tasks down into categories that capture how different each question's response is and whether the code can be fully executed by Python (denoted Python only vs. Python + LM). For some tasks within the benchmark, each question has the same code or Chain of Thought, with the only variation being the inputs – in this case we say the code is (repeated code), and if not then it is denoted (new code).

Table 1. Overall performance (%) on BIG-Bench Hard with both few-shot prompting with a single task and cross-task. The delta compared to direct prompting is shown in parenthesis.

		text-davinci-003			PaLM 2-S* (code variant (Google et al., 2023))		
Prompt	Human	Direct	CoT	CoC (Ours)	Direct	CoT	CoC (Ours)
Single task	68	55	72 (+17)	84 (+29)	49	61 (+12)	78 (+29)
Cross task	-	50	55 (+5)	61 (+11)	45	47 (+2)	47 (+2)

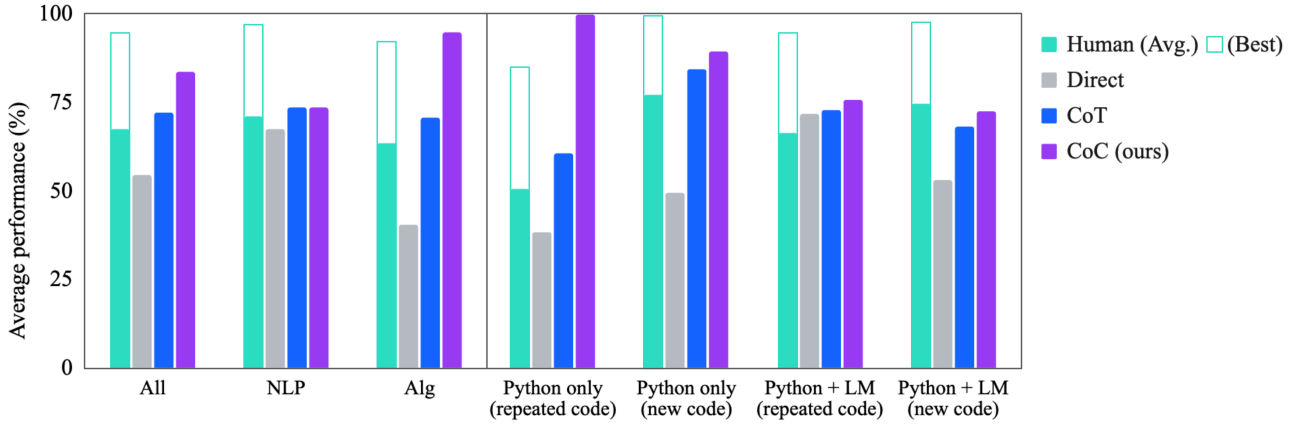

Figure 4. Average performance across different baselines grouped by task type, indicating the problem type and how CoC is generated & executed.

As expected, we see that when the code is repeated and run by Python, CoC gets nearly 100%, though these tasks (e.g., multi-step arithmetic) seem to be among the most challenging for the other baselines, including human raters. The other categories are more challenging for CoC; however in each, we still see a benefit over baselines.

Question 3: Ablations. Figures 5 and 6, and Table 2 show the ablations performed to motivate each aspect of Chain of Code prompting. As one may expect, the approaches that execute Python (CoC (Interweave, Python, try Python except LM, try Python except LM state)) achieve 100% performance on several tasks – if the code is correct, then the model will be correct every time. However, the approach that relies on Python alone (CoC (Python)) performs poorly when applied to non-algorithmic tasks, failing almost all. The CoC (Python) ablation is similar to recent works (Gao et al., 2023; Chen et al., 2022), which show that if applied to numerical problems then code reasoning performs well. CoC without the Python interpreter (CoC (LM, LM state)) too fares poorly, though we see that the step-by-step approach proposed in ScratchPad prompting (Nye et al., 2021) improves in each task.

We also show that ablations CoC (try Python except LM, try Python except LM state), in which CoC first tries to run the entire code with Python and if it fails simulates the code with an LM, perform quite well. Again we see that maintaining a program state provides an improvement in performance. With only minor degradations in performance observed, they are reasonable alternatives to the fully interweaved CoC for their simplicity. Though we note, these ablations' performance would be much worse in cases where interweaving code and semantics is truly necessary – for example, if we imagine a case where code is necessary to parse image inputs or to access an external database, but language is necessary to parse the results (see the robotics applications in Section A.6).

Question 4: Scaling. Figure 7 shows the performance of CoC across various model sizes. We observe that, similar to Chain of Thought prompting, the improvements of CoC increases as model size increases. In fact, for some of the algorithmic tasks, Chain of Code even outperforms the best human raters (whom admittedly did not have access to code). Unlike Chain of Thought prompting, however, which only brings performance benefits for the largest model (d-3), CoC outperforms the direct question answering baseline also for smaller models (a-1, b-1, c-1), suggesting that it's easier for smaller models to output structured code as intermediate steps rather than natural languages.

Question 5: Cross-task Prompting. For cross-task prompting, we prompt the language models with a few examples from different problems. We see the performance drops for all methods in Figure 7 and Table 1. Despite this drop, CoC outperforms CoT and direct prompting at scale, nearly achieving human average performance. This is a promising indication towards general purpose reasoning, in which a model does not expect to receive examples of similar problems in its prompt.

Question 6: Instruction Tuned Models. The reason why we chose `text-davinci-003`, a completion model, as our primary evaluation model, over more advanced instruction tuned models (`gpt-3.5-turbo` and `gpt-4`) is that the